SOFTWARE TESTING

Week 6

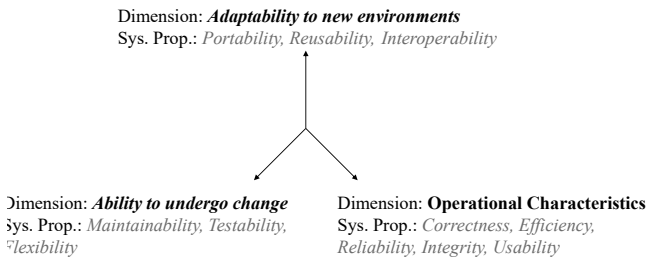
Software Quality

- ***Software Quality***: Conformance to
  - ***explicitly stated*** *functional and non-functional (performance, etc) requirements*,
  - ***explicitly documented*** *development standards*, and
  - *implicit characteristics* that are expected of all ***professionally*** *developed software*.

Software Quality

- *Standard Glossary of SW Engineering Terminology* [IEEE610.12]:
  Quality: (1) The degree to which a system, component, or process *meets* ***specified requirements***. (2) The degree to which a system, component, or process *meets customer or* ***user needs or expectations***.

Dimensions of SW Quality



System Properties

- ***Portability***: The effort required to transfer the system from one hardware and/or software environment to another.
- ***Reusability***: The extent to which the system (or part of it) can be reused in other applications.
- ***Interoperability***: The effort required to couple the system to another.
- ***Maintainability***: The effort required to introduce a modification (usually a correction) into the system.
- ***Flexibility***: The effort required to modify or customize the system in operation.
- ***Testability***: The effort required to test the system to ensure that it performs its intended function.

System Properties

- ***Correctness***: The extent to which the system satisfies its specification and fulfills the users' needs.
- ***Reliability***: The extent to which the system can be expected to perform its intended function with required precision and without failure.
- ***Efficiency***: The amount of computing resources (space, time) required by the system to perform its function.
- ***Integrity***: The extent to which access to the system or its data by unauthorized persons can be controlled.
- ***Usability***: The effort required to learn, operate, prepare input and interpret output of the system.

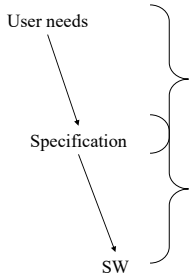
Software Quality

- *Standard Glossary of SW Engineering Terminology* [IEEE610.12]:
  Quality assurance: (1) A planned and systematic pattern of all actions necessary to provide *adequate confidence* that an item or product conforms to established technical requirements. (2) A set of activities designed to *evaluate the process* by which products are developed or manufactured.

Quality Assurance

- application of sound technical methods and tools
- formal technical reviews and inspections
- ***software testing***
- enforcement of standards
- documentation
- control of change
- extensive measurement
- record keeping and reporting of the process

Verification & Validation



Validation: Are we ***building the right product?***

Verification: Are we ***building the product right?***

Principles of SW Testing

1. *Testing* is a process of *executing a program with the intent of finding a defect*. So, there must be some program code to be executed.
2. A *good test case* is one that has a high probability of *finding an as yet undiscovered defect*. So, the test cases (the program input) should be selected systematically and with care, both for correct and incorrect behavior.
3. A *successful test is one that uncovers an as yet undiscovered defect*. So, testing is psychologically ***destructive*** since it tries to demolish the software that has been constructed.

Principles of SW Testing

4. Testing cannot show the absence of defects, it can only show that they are present (Dijkstra). [Testing is not *formal verification*.]
5. Testing is quite an ineffective method of quality assurance. [Though, usually the most applicable one.]
6. Successful testing shall be *followed by* a separate *debugging* phase.
7. Testing is also by itself a *process* that must be systematically managed (and assisted with special testing tools).

Facts of Testing

- *V&V engineer must define "correct" and "incorrect."*
- There must be *a specification against which to check the results of testing*.
- Full automation of testing is impossible:
  - in theory, the total behavior of a program is undecidable (halting, failures);
  - in practice, exhaustive testing is intractable;
  - tracking of (technical) failures to (human) errors is impossible;
  - we can never be sure that the testing tool (a program) works correctly.

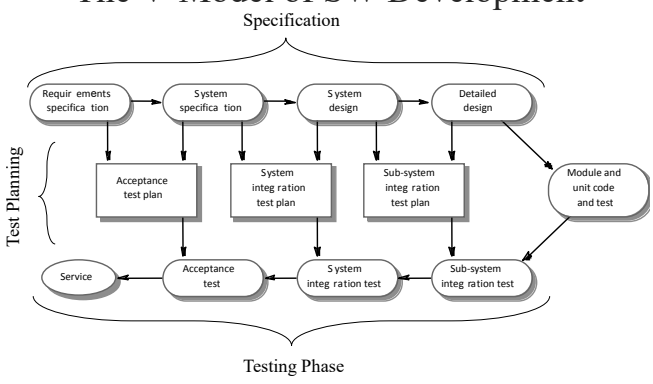
An Example

- The program reads three integer values. The three values are interpreted as representing the lengths of the sides of a triangle. The program prints a message that states whether the triangle is scalene, isosceles, or equilateral. *Write test cases* (*specific input values*) *that you feel would adequately test this program.*
- In a valid triangle, no side may have a length of zero or less, and each side must be shorter than the sum of all sides divided by 2.
- ***Equilateral*** (eşkenar) triangle: all sides are of equal length.
- ***Isosceles*** (ikizkenar) triangle: two sides are of equal length.
- ***Scalene*** (çeşitkenar) triangle: all sides are of unequal length.

Some of Possible Test Cases [1]

1. (5, 3, 4): scalene
2. (3, 3, 4): isoscele
3. (3, 3, 3): equilateral
4. (50, 50, 25): isoscele
5. (25, 50, 50): isoscele (permutation)
6. (50, 25, 50): isoscele (permutation)
7. (10, 10, 0): invalid (zero)
8. (3, 3, -4): invalid (negative)
9. (5, 5, 10): invalid (too long)
10. (10, 5, 5): invalid (too long, perm.)
11. (5, 10, 5): invalid (too long, perm.)
12. (8, 2, 5): invalid (Too long)
13. (2, 5, 8): invalid (Too long, perm.)
14. (2, 8, 5): invalid (Too long, perm.)
15. (8, 5, 2): invalid (Too long, perm.)
16. (5, 8, 2): invalid (Too long, perm.)
17. (5, 2, 8): invalid (Too long, perm.)
18. (0, 0, 0): invalid (all zeros)
19. (@, 4, 5): invalid (non-integer)
20. (3, $, 5): invalid (non-integer)
21. (3, 4, %): invalid (non-integer)
22. (, 4, 5): invalid (missing input)
23. (3,,5): invalid (missing input)
24. (3, 4,): invalid (missing input)

The V-Model of SW Development



Jargon of Testing

- **Error (yanlışlık):** A mistake (*human* action) made by a software developer. It might be a total misinterpretation of user requirements, or a simple typograhpical misprint. An error introduces a defect into the *software code.*
- **Defect, fault, bug (arıza, sorun):** A difference between the incorrect program and its correct version; a coding error. A defect in the software, if encountered during *execution*, *may cause a failure*.
- **Failure (başarısızlık)**: An externally observable deviation of the functional software from its specification; an *incorrect result of computation*.

Why testing? Why *systematic* testing?

- According to several empirical studies, a (*professionally produced commercial*) SW system...
  - ...contains ***3 – 30 defects per 1000 lines of code***
  - ..., the average debugging effort is ***12 hours of working time per defect***
  - ..., ***maintenance*** is about *half of software development costs*, mostly in error removal

Example... cont'd

- In mathematics, the number of integer values is infinite. However, computers have finite space which limits the number of values that can be processed. Let us assume that our triangle program is running in a tiny computer with 10.000 as the largest integer value. Then there are $10^4 * 10^4 * 10^4 = 10^{12}$ possible length combinations of triangle sides (including the invalid ones).
- Suppose you are a very fast tester, running and checking 1000 tests per second, 24 hours per day, 365 days per year.
- Then the exhaustive testing effort (testing each possible length combination) would take over 31.7 years.

Some Remarks

- *most test cases represent invalid inputs*
- each valid triangle type is tested at least once
- permutations are used to check that the order of the input values does not affect the result
- boundary input values are used (length of exactly zero, length of exactly the sum of all sides divided by 2)
- input values of wrong type (non-integers) are used
- the number of test cases is rather small with respect to the number of all possible inputs

Module (unit) testing:

- each independent unit tested separately
- level: *source code*
- need for simulated execution environment

Integration testing:

- *modules grouped into subsystems* for testing
- *"big bang:"* all the modules tested as a whole
- *incremental approaches: modules into subsystems*
- level-wise ("stubs" and "drivers")
- level: interfaces between modules

System Testing

- the *whole system* (including hardware, databases, sensors, …) *tested*
- target: performance, capacity, fault-tolerance, security, configuration, … (non-functional req.s)
- level: external interface

Special Types of Testing

- Volume testing
- Load / stress testing
- Security testing
- Performance testing
- Configuration testing
- Installability testing
- Recovery testing
- Reliability / availability testing
- Maintainability testing
- Protocol conformance testing, etc.

Volume Testing

- a *non-functional* test.
- refers to *testing a software application with a certain amount of data*
  - the ***database size*** or
  - the size of an interface file that is the subject of volume testing.
- Examples (for *performance* testing)
  - volume testing an application with a specific database size,
    - expand your database to that size and then test the application's ***performance*** on it.
  - suppose a requirement for your application is to interact with an interface file (could be any file such as .dat, .xml);
  - this interaction could be reading and/or writing on to/from the file. You will create a sample file of the size you want and then test the application's functionality with that file in order to test the performance.

Security Testing (ST)

- determines that a SW product protects data and maintains functionality as intended.
- *six basic concepts* to be covered by ST
  - *Confidentiality*: protection against disclosure of info to unauthorized parties
  - *Integrity*: system permission that receiver can determine information she receives is correct.
  - *Authentication*: confirmation identity of a person, tracing origins of an artifact or ensuring that a product is what its packaging claims it is, or making sure a computer program is a trusted one.
  - *Authorization*: determining that a requester is allowed to receive a service or perform an operationç
  - *Availability*: Assuring that relevant service will be ready for use when expected
  - *Non-repudiation*: assuring that some message transferred is sent and received by the parties claiming to have sent and received it.

Performance Testing

- Determines how a system performs in terms of *responsiveness* and *stability* under a particular workload.
- Also investigates, measures, validates or verifies *other system properties such as scalability, reliability and resource usage*.

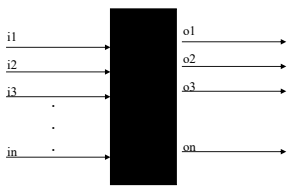
Acceptance Testing

- user involvement (alpha, beta)
- "actual needs" tested
- ***usability testing*** at the user interface
- development team in development environment:
  - "standard", general usability errors
- real user representatives in laboratory environment:
  - task-specific usability problems (real tasks, talk-aloud, taping, post-analysis by experts)

Black-box (*functional*) *testing*

- *internal details* of modules or subsystems are *hidden* and cannot be studied from outside
- concentrates on the *interfaces* of *modules and (sub)systems* (e.g. user interface)
- externally observable functionality and input-output behavior
- based on input *classification*
- especially suitable for *integration*, *system*, and *acceptance testing*

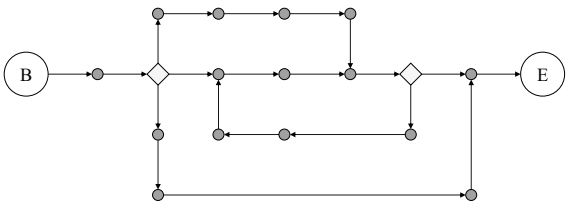
Black-box (*functional*) *testing*



White-box (*structural*) *testing*

- structure of the software is examined in detail at the level of program code
- objective to traverse as many *paths* over the code as considered necessary
- based on *control flow* and *data flow*
- several forms of *coverage* (path, statement, branch, …)
- especially suitable for *module* (*unit*) testing

White-box (structural) testing

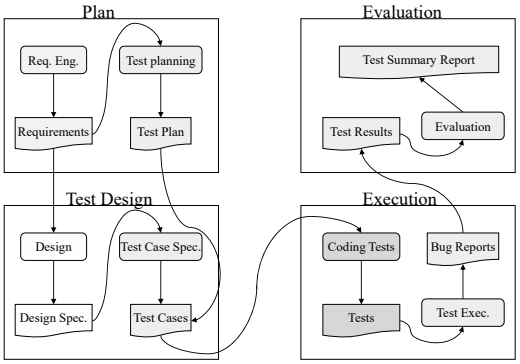


Management of Testing

- *Plan*
- *Execute*
- *Evaluate*
- *Document*
- *Report*

process

Testing Process



Standard for Software Test Documentation [IEEE 829]

1) Test plan: the scope, approach, resources, and schedule of the testing activities.

2) Test-design specification: the refinements of the test approach, and the features to be tested by the design and its associated tests.

3) Test-case specification: a test case identified by a test-design specification.

4) Test-procedure specification: the steps for executing a set of test cases or, more generally, the steps used to analyze a software item in order to evaluate a set of features.

Standard for Software Test Documentation [IEEE 829]

5) Test-item transmittal report: the test items being transmitted for testing, including the person responsible for each item, its physical location, and its status.

6) Test log: a chronological record of relevant details about the execution of tests.

7) Test-incident report (***bug report***)**:** any event that occurs during the testing process which requires investigation.

8) Test-summary report: the results of the designated activities, and evaluations based on these results.

Test plan

1) Test-plan identifier: specifies the unique identifier assigned to the test plan.

2) Introduction: summarizes the software items and software features to be tested, provides references to the documents relevant for testing (overall project plan, quality assurance plan, configuration management plan, applicable standards…).

3) Test items: identifies the items to be tested, including their version/revision level; provides references to the relevant item documentation (requirements specification, design specification, user's guide, operations guide, installation guide, …); also identifies items which are specifically excluded from testing.

Test Plan

4) Features to be tested: identifies all software features and their combinations to be tested, identifies the testdesign specification associated with each feature and each combination of features.

5) Features not to be tested: identifies all features and significant combinations of features which will not be tested, and the reasons for this.

6) Approach: describes the overall approach to testing (the testing activities and techniques applied, the testing of non-functional requirements such as performance and security, the tools used in testing); specifies completion criteria (for example, error frequency or code coverage); identifies significant constraints such as testing-resource availability and strict deadlines; serves for estimating the testing efforts.

Test Plan

7) Item pass/fail criteria: specifies the criteria to be used to determine whether each test item has passed or failed testing.

8) Suspension criteria and resumption: specifies the criteria used to suspend all or portion of the testing activity on the test items (at the end of working day, due to hardware failure or other external exception, …), specifies the testing activities which must be repeated when testing is resumed.

9) Test deliverables: identifies the deliverable documents, typically test-design specifications, test-case specifications, test-procedure specifications, test-item transmittal reports, test logs, test-incident reports, description of test-input data and test-output data, description of test tools.

Test Plan

10) Testing tasks: identifies the set of tasks necessary to prepare and perform testing (description of the main phases in the testing process, design of verification mechanisms, plan for maintenance of the testing environment, …).

11) Environmental needs: specifies both the necessary and desired properties of the test environment (hardware, communications and systems software, software libraries, test support tools, level of security for the test facilities, drivers and stubs to be implemented, office or laboratory space, …).

Test Plan

12) Responsibilities: identifies the groups of persons responsible for managing, designing, preparing, executing, witnessing, checking, and resolving the testing process; identifies the groups responsible for providing the test items (section 3) and the environmental needs (section 11).

13) Staffing and training needs: specifies the number of testers by skill level, and identifies training options forproviding necessary skills.

14) Schedule: includes test milestones(those defined in the overall project plan as well as those identified as internal ones in the testing process), estimates the time required to do each testing task, identifies the temporal dependencies between testing tasks, specifies the schedule over calendar time for each task and milestone.

Test Plan

15) Risks and contingencies: identifies the high-risk assumptions of the test plan (lack of skilled personnel, possible technical problems, …), specifies contingency plans for each risk (employment of additional testers, increase of night shift, exclusion of some tests of minor importance, …).

16) Approvals: specifies the persons who must approve this plan.

Test-case specification

4) *Output specifications*: specifies all of the outputs and features (for example, response time) required of the test items, provides the exact value (with tolerances where appropriate) for each required output or feature.
5) *Environmental needs*: specifies the hardware and software configuration needed to execute this test case, as well as other requirements (such as specially trained operators or testers).
6) *Special procedural requirements*: describes any special constraints on the test procedures which execute this test case (special set-up, operator intervention, …).
7) *Intercase dependencies*: lists the identifiers of test cases which must be executed prior to this test case, describes the nature of the dependencies.

Test-incident report (bug report)

3) *Bug description*: provides a description of the incident, so as to correct the bug, repeat the incident or analyze it off-line:
   a. Inputs.
   b. Expected results.
   c. Actual results.
   d. Date and time.
   e. Test-procedure step.
   f. Environment.
   g. Repeatability (whether repeated; whether occurring always, occasionally or just once).
   h. Testers.
   i. Other observers.
   j. Additional information that may help to isolate and correct the cause of the incident; for example, the sequence of operational steps or history of user-interface commands that lead to the (bug) incident.
4) *Impact*: Priority of solving the incident / correcting the bug (urgent, high, medium, low).

Test-summary report

3) *Variances*: reports any variances/deviations of the test items from their design specifications, indicates any variances of the actual testing process from the test plan or test procedures, specifies the reason for each variance.
4) *Comprehensiveness assessment*: evaluates the comprehensiveness of the actual testing process against the criteria specified in the test plan, identifies features or feature combinations which were not sufficiently tested and explains the reasons for omission.
5) *Summary of results*: summarizes the success of testing (such as coverage), identifies all resolved and unresolved incidents.

Inspection checklist for test plans:

1) Have all materials required for a test plan inspection been received?
2) Are all materials in the proper physical format?
3) Have all test plan standards been followed?
4) Has the testing environment been completely specified?
5) Have all resources been considered, both human and hardware/software?
6) Have all testing dependencies been addressed (driver function, hardware, etc.)?
7) Is the test plan complete, i.e., does it verify all of the requirements? *For unit testing*: does the plan test all functional and structural variations from the high-level and detailed design?)
8) Is each script detailed and specific enough to provide the basis for test case generation?
9) Are all test entrance and exit criteria sufficient and realistic?
10) Are invalid as well as valid input conditions tested?
11) Have all pass/fail criteria been defined?

Test-case specification

1) *Test-case-specification identifier*: specifies the unique identifier assigned to this test-case specification.
2) *Test items*: identifies and briefly describes the items and features to be exercised by this test case, supplies references to the relevant item documentation (requirements specification, design specification, user's guide, operations guide, installation guide, …).
3) *Input specifications*: specifies each input required to execute the test case (by value with tolerances or by name); identifies all appropriate databases, files, terminal messages, memory resident areas, and external values passed by the operating system; specifies all required relationships between inputs (for example, timing).

Test-incident report (bug report)

1) *Bug-report identifier*: specifies the unique identifier assigned to this report.
2) *Summary*: summarizes the (bug) incident by identifying the test items involved (with version/revision level) and by referencing the relevant documents (test procedure specification, test-case specification, test log).

Test-summary report

1) *Test-summary-report identifier*: specifies the unique identifier assigned to this report.
2) *Summary*: summarizes the evaluation of the test items, identifies the items tested (including their version/revision level), indicates the environment in which the testing activities took place, supplies references to the documentation over the testing process (test plan, test-design specifications, test-procedure specifications, test-item transmittal reports, test logs, test-incident reports, …).

Test-summary report

6) *Evaluation*: provides an overall evaluation of each test item including its limitations (based upon the test results and the item-level pass/fail criteria).
7) *Summary of activities*: summarizes the major testing activities and events, summarizes resource consumption (total staffing level, total person-hours, total machine time, total elapsed time used for each of the major testing activities, …).
8) *Approvals*: specifies the persons who must approve this report (and the whole testing phase).

Inspection checklist for test plans:

12) Does the test plan outline the levels of acceptability for pass/fail and exit criteria (e.g., defect tolerance)?
13) Have all suspension criteria and resumption requirements been identified?
14) Are all items excluded from testing documented as such?
15) Have all test deliverables been defined?
16) Will software development changes invalidate the plan? (*Relevant for unit test plans only*.)
17) Is the intent of the test plan to show the presence of failures and not merely the absence of failures?
18) Is the test plan complete, correct, and unambiguous?
19) Are there holes in the plan; is there overlap in the plan?
20) Does the test plan offer a measure of test completeness and test reliability to be sought?
21) Are the test strategy and philosophy feasible?

Inspection checklist for test cases:

1) Have all materials required for a test case inspection been received?
2) Are all materials in the proper physical format?
3) Have all test case standards been followed?
4) Are the functional variations exercised by each test case required by the test plan? (*Relevant for unit test case documents only*.)
5) Are the functional variations exercised by each test case clearly documented in the test case description? (*Relevant for unit test case documents only*.)
6) Does each test case include a complete description of the expected input, and output or result?
7) Have all testing execution procedures been defined and documented?
8) Have all testing dependencies been addressed (driver function, hardware, etc.)?
9) Do the test cases accurately implement the test plan?

References

[1] Myers, *The Art of Software Testing*, 1978

Appendix A: Drivers and Stubs

- ***Idea***: develop and test software in “pieces” (modular approach)
- ***Problem***: how to test a "piece" if the other "pieces" that it uses have not yet been developed (and vice versa).
- ***Solution***: stubs and drivers.

- ***Relevance***: White-box testing
  - must run the code with predetermined input and check to make sure that the code produces predetermined outputs.
  - Often testers write stubs and drivers for white-box testing.

Appendix A: Drivers

Driver for Testing:

Driver is a the piece of code that passes test cases to another piece of code. Test Harness or a test driver is supporting code and data used to provide an environment for testing part of a system in isolation. It can be called as as a software module which is used to invoke a module under test and provide test inputs, control and, monitor execution, and report test results or most simplistically a line of code that calls a method and passes that method a value.

For example, if you wanted to move a fighter on the game, the driver code would be moveFighter(Fighter, LocationX, LocationY);

This driver code would likely be called from the main method. A white-box test case would execute this driver line of code and check "fighter.getPosition()" to make sure the player is now on the expected cell on the board.

Appendix A: Stubs

A Stub is a dummy procedure, module or unit that stands in for an unfinished portion of a system.

Four basic types of Stubs for Top-Down Testing are:

1 Display a trace message
2 Display parameter value(s)
3 Return a value from a table
4 Return table value selected by parameter

A stub is a computer program which is used as a substitute for the body of a software module that is or will be defined elsewhere or a dummy component or object used to simulate the behavior of a real component until that component has been developed.

Ultimately, the dummy method would be completed with the proper program logic. However, developing the stub allows the programmer to call a method in the code being developed, even if the method does not yet have the desired behavior.

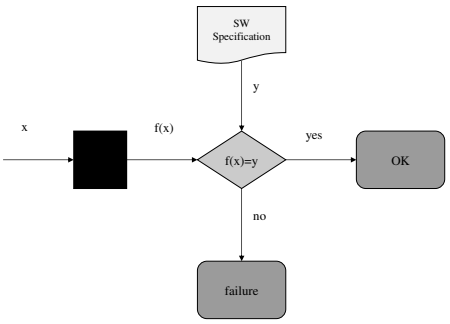
Stubs and drivers are often viewed as throwaway code. However, they do not have to be thrown away: Stubs can be "filled in" to form the actual method. Drivers can become automated test cases.

- Integrity ...
  - ... are addressed in processes to source and create SW components and deliver them to customers
  - ... contain controls to enhance confidence that SW was not modified without consent of supplier.

BLACKBOX TESTING

Week 9

Black Box Testing



Principles

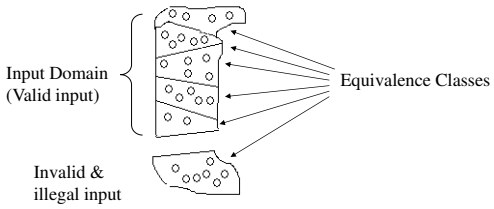
- Based on ***specifications*** and ***documents***
  - *requirements*
  - *technical plans, architectures*
  - *user manuals*
- *Code not necessarily needed* (while it certainly helps)
- General strategy; applies especially to
  - *integration testing, system testing, acceptance testing*
- Can be assisted by a post-white-box testing phase, to obtain code coverage measures as indicators of testing quality

Domain partitioning:Equivalence classes

- *System domain*: set of *all input values*

- *Equivalence class*: *certain set of input values* (subset of domain, *subdomain*)

Equivalence Classes (ECs)



Equivalence Classes

- Each EC *represents* a *central property* of system
- each value in an EC makes system *behave "in the same manner*"
  - in testing, ***each*** value reveals a *failure* or makes system behave *ok*
- each value activates (almost) the same execution path through the system
- based on
  - ***system's specification*** and
  - ***experience / intuition of tester***

Black-box testing hypothesis

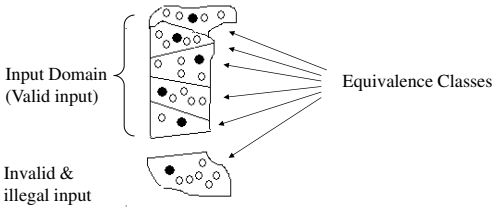
- each value in an EC results in
  - *correct execution*, or
  - *failure*

  when used as input to system
- *for testing purposes*, ***one*** representative input value from each EC is enough!
- ***in practice***, the hypothesis does not hold universally, so ***system shall be tested with several input values from each EC***.

Equivalence Classes

- Each "*black dot*" represents the equivalence class it is in.
- Testing the code using a black dot will result either
  - in a failure or
  - OK

and represent the entire equivalence class.



Forming equivalence classes (ECs)

- To specify: a ***range of values***
- Corresponding ECs: ***one valid and two invalid classes***
- ***Example 1:*** "$a \le x \le b$, x an integer"
  - Valid EC: {integer $x \mid a \le x \le b$},
  - Invalid EC: {integer $x \mid x < a$},
  - Invalid EC: {integer $x \mid x > b$}
- To specify: a ***specific value within a range***
- Corresponding ECs: ***one valid and two invalid classes***
- ***Example 2:*** "value of integer x shall be t"
- Valid EC: {integer $x \mid x = t$},
- Invalid EC: {integer $x \mid x < t$},
- Invalid EC: {integer $x \mid x > t$}

Forming equivalence classes (ECs)

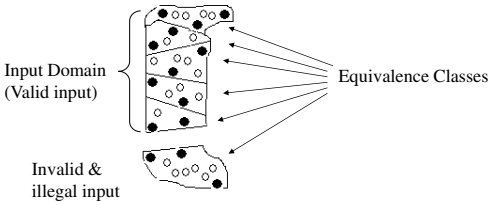
- To specify: a ***set of values***
- Corresponding ECs: ***one valid and one invalid classes***
- ***Example 3:*** "2D geometric shape x shall have *4 corners*"
  - Valid EC: $x \in$ {square, rectangle, trapezoid, parallelogram, ...},
  - Invalid EC: $x \in$ {ellipsoid, circle, triangle, pentagon, hexagon, heptagon, ...},
- To specify: a ***boolean value***
- Corresponding ECs: ***one valid and one invalid classes***
- ***Example 4:*** "x shall be true"
  - Valid EC: x = *true*
  - Invalid EC: x = *false*

Forming equivalence classes (ECs)

- one or more ECs for ***illegal*** values (i.e., values incompatible with the type of the input parameter and therefore out of the parameter's domain
- ***Example:*** "integer values x"
  - Illegal EC: real-number x
  - Illegal EC: character-string x
- ***How many ECs?***
  - As many as the *potential groups of values* that are believed to be *handled by the system in different ways*.
  - Any EC shall be further divided into subclasses if there is reason to believe values in different subclasses are not processed by the system identically.

Boundary analysis

- EC boundaries are where bugs critically show up. That's why boundary conditions are subject to test.
- Each "*black dot*" represents a boundary condition of its relevant EC it is in.



Boundary Conditions

- ***open boundaries***: generated by inequality operators (<, >)
- ***closed boundaries***: generated by equality operators (=, ≤, ≥)
- ***on point***: value that lies on a boundary
  - for open boundaries: the boundary value; for instance $x > 0$
- ***off point***: value not on a boundary
- "one-by-one" *domain testing strategy*: one on point and one off point for each domain boundary

Selection rules for on and off points:

- *open boundary*: ***one on point and one off point***
  - ***on point:*** a value outside the domain ⇒ the condition is *false*
  - ***off point:*** a value inside the domain ⇒ the condition is *true*
- *closed boundary*: ***one on point and two off points*** (on both sides of the boundary, as close as possible)
  - ***on point:*** a value inside the domain ⇒ the condition is *true*
  - ***off point:*** a value outside the domain ⇒ the condition is *false*
- *nonscalar type*: ***one on point and one off point***
  - enumerations, Booleans, strings, complex numbers, …
  - on point: the condition is *true*
  - off point: the condition is *false*
  - the difference between on and off values should be minimized (for instance, for strings a single character difference)

Examples

- ***range of values:*** two boundary conditions
- ***"integer x shall be between a and b"*** ⇒ {integer x | (x ≥ a)∪(x ≤ b)}: (x ≥ a), (x ≤ b) are closed boundaries
  - ***on points:*** *a, b*
  - ***off points:*** *a-1, a+1, b-1, b+1*
- ***strict inequality operator*** ⇒ open subdomain
  "integer *x* shall be greater than *a*" ⇒ {integer *x* | *x* > *a*}
  - ***on point:*** *a*
  - ***off point:*** *a+1*

Examples

- **specific *value:*** one closed boundary condition
  - "value of integer *x* shall be *a*" ⇒ {integer *x* | *x* = 100}
  - ***on point:*** *a*
  - ***off points:*** *a-1, a+1*
- ***set of values*** ⇒ ***nonscalar type***
  - "weekday *x* shall be a working day" ⇒
  - *x* ∈ {Monday, Tuesday, Wednesday, Thursday, Friday}
  - ***on point:*** Friday, ***off point:*** Saturday
- ***Boolean*** ⇒ ***nonscalar type***
  - ***on point:*** *true*, ***off point***: *false*

The category-partition method

- *systematic black-box test design method*
- *based on equivalence partitioning* of input.
- ***Steps***
  1. **Specification of input *categories or* "*problem parameters*"**
  2. **Division of categories into *choices* = equivalence classes**
  3. ***Test specification*:**
  4. **Generation of test cases for the test frames into executable form (using a tool), combination into *test suites*.**
  5. Storing the testware into a *test database.*
  6. **Testing of the unit by the test cases, refinement of conflicting choices, maintenance of test database (using a tool).**

Array Sorting Example: Steps

i. Specification of input *categories or* "*problem parameters*"

- ***Array sorting categories:***
  - *size of array*
  - *type of elements*
  - *maximum element value*
  - *minimum element value*
  - *position of maximum element in the array*
  - *position of minimum element in the array*

Step 2: Division of Categories

ii. Division of categories into *choices* = equivalence classes

- ***Array sorting / choices for size of array:***
  - *size* = 0
  - *size* = 1
  - 2 ≤ *size* ≤ 100
  - *size* > 100
  - ("*size* is illegal")

Step 3:Test Specification

***iii. Test specification*:**

- A set of *test frames*: sets of choices, with each category contributing either zero or one choice.
- A set of *test cases*: a single value from each of the choices in a test frame.
- ***Array sorting example*** / test case:
  - *size of array* = 50 (choice: 2 ≤ *size* ≤ 100)
  - *type of elements* = integer
  - *maximum element value* = 91
  - *minimum element value* = -3
  - *position of maximum element in the array* = 15
  - *position of minimum element in the array* = 43

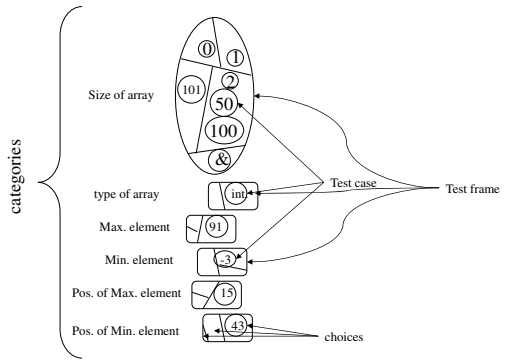
The category-partition method

(4) ***Generation of test cases for the test frames into executable form*** (using a tool), combination into *test suites*.

(5) Storing the testware into a *test database*.

(6) ***Testing of the unit*** by the test cases, refinement of conflicting choices, maintenance of test database (using a tool).

Example



System testing / GUI testing:

- ***target:*** *operations* available at the (graphical) user interface
- ***parameters* of operations** divided *into equivalence classes*
- testing by all different *combinations* of equivalence classes (with one input value from each class)
- **testing of operation *sequences*** (not independent)
- based on user's manual
- supported by tools (capture / replay)

Example: Find String in Document

- ***Find*** **(document, text, direction, match case)**
- *document*: the current text file, subject to search
- *text*: the character string to search for
- *direction* (*down*, *up*): direction of the search with respect to current position of the cursor
- *match case* (*yes*, *no*): whether or not the operation is case sensitive to letters

Equivalence classes

- **Input categories for various input**
- *text*:
- {strings with lower-case letters but without upper-case letters}
- {strings with upper-case letters but without lower-case letters}
- {strings with both upper-case and lower-case letters}
- {strings with no letters}
- {empty (illegal) strings}
- *direction*: {*down*}, {*up*}
- *match case*: {*yes*}, {*no*}
- *document*: {text found}, {text not found}

Example

text					drctn		c. mtch		dcmnt	
lc	uc	luc	nlu	ε	d	u	n	y	f	n-f
☺					☺			☺	☺	
☺					☺			☺		☺
☺						☺		☺	☺	
☺						☺		☺		☺
...										
	☺									
	☺									
	...									
				☺	☺		☺			☺

How many tests?

- ***# of (independent) combinations = Total number of tests***
- $E_1 * E_2 * E_3 * \ldots * E_k$
  - with E_i = # equivalence classes for parameter *i*
- ***For find example: 5 * 2 * 2 * 2 = 40 tests***
- Some invalid, illegal combinations that might be unexecutable must be tested too!

Test Case Patterns

- *text*: lower-case, *direction*: down, *match case*: yes, *document*: found **(1)**
- *text*: lower-case, *direction*: down, *match case*: yes, *document*: ***not found*** **(2)**
- *text*: lower-case, *direction*: ***up***, *match case*: yes, *document*: found **(3)**
- *text*: lower-case, *direction*: up, *match case*: yes, *document*: ***not found*** **(4)**
- **…**
- *text*: ***empty***, *direction*: up, *match case*: no, *document*: not found **(40)**

Selection of test cases (40):

- each pattern generates a test case
- each equivalence class in a pattern is realized as an input value in the corresponding test case
- in different test cases, different values are selected for the same equivalence class (better coverage)
- boundary values are selected, when applicable
  - for text, both short and long character strings
  - for text, the whole character set

Test cases - 1

document		text	direction	Match case
This **bea**utiful text	1	bea	down	yes
This beautiful text	2	beatles	down	yes
This **1bea**utiful text	3	1bea	up	yes
This 1Beautiful text	4	1bea	up	yes
This &**%1bEAu**tiful text	5	%1beau	down	no
This &%2beautiful text	6	%1beau	down	no
This BE utiful text	7	b	up	no
This BE utiful text	8	beauti	up	no
This **BEA**UTIFUL text	9	BEA	down	yes

Test cases - 2

document		text	direction	Match case
This BEAUTIFUL text	10	BEAT	down	yes
THIS beautiFUL text	11	THIS	up	yes
THIS beatiful text	12	T2S	up	yes
T**his** Beautiful Text	13	HIS	down	no
this %#& beautiful text	14	S	down	no
this %#& beautiful text	15	HIS%#&	up	no
This %#&beautiful text	16	#& BE	up	no
This Beautiful **Text**	17	Text	down	yes
This Beautiful Text	18	Text	down	yes

Test cases - 3

document		text	direction	Match case
TH**IS is** beautiful text	19	IS is	up	yes
This is beautiful text	20	IS is	up	yes
This t**ext 1**-99	21	ExT 1	down	no
This text 1 and text 2	22	eXt 1	down	no
T**his was beauti**ful text	23	His Was Beauti	down	no
(This) (Was) (123text)	24	aS()	up	no
123 one-two-three	25	123	down	yes
One-two-three 1-2-3	26	12-3	down	yes
This **&**007# mess	27	&	up	yes

Test cases - 4

document		text	direction	Match case
This Bloody Mess	28	#%	up	yes
(This) (was1) (was[**2**])	29	2]	down	no
0987654321!"#%&/*///	30	7654321#	down	no
1!2"3**#4$5%6&7/8(9)**0=oops		#4$5%6&7/8(9)	up	no
This %#&beautiful text	32	22	up	no
This is beautiful texT	33		down	no
1 or two	34		down	yes
1 or two	35		up	yes
0K1+(8Those	36		up	yes
1 & 2	37		down	no

Test cases - 5

document		text	direction	Match case
	38		down	no
This %#&beautiful text	39		up	no
	40		up	no

Example

- ***print (file, copies, font, pagination)***
- **Input parameters**:
  - *name of the file* ***(must be provided)***
  - *-cn*, where n is the number of copies ($1 \le n \le 100$);
    - default: n = 1
  - *-fkm*, where k indicates a font ($1 \le k \le 9$) and m indicates a mode (N for normal or B for bold);
    - defaults: k = 1, m = N
  - *-np*: no pagination (default: pagination shall be done)

Example... *Equivalence classes*

- *Originating from **file name***:
  1. Name of existing file given (*V*alid).
  2. No file name given (*Not**V**alid*).
  3. Name of non-existing file given (*NV*).
  4. "Name" does not follow the syntactic rules (*NV*).
- *Originating from **copies (-cn)***:
  5. $1 \le n \le 100$ (*V*).
  6. Default: no n given (*V*).
  7. n = 0 or n > 100 (*NV*).

Example... *Equivalence classes*

- *Originating from **fonts (-fkm)***:
  8. $1 \le k \le 9$ (*V*).
  9. Default: no *k* given (*V*).
  10. m = N or m = B (*V*).
  11. Default: no m given (*V*).
  12. k = 0 or k > 9 (*NV*).
  13. m other than N or B (*NV*).
- *Originating from **pagination (-np)***:
  14. *-np* given (*V*).
  15. *-np* not given (*V*).
  16. Something else than *-np* given (*NV*). (This class covers also the other syntactically invalid *-options*.)

Example... Number of exhaustive combinatory test cases

print *file* **[-c*n*] [-f *k m*] [-*np*]**

4 * 3 * 3 * 3 * 3 = **324** test cases

This might be too many, so a method reducing the number of test cases is needed.

Optimizing Principle

- ***print file* [-c*n*] [-f*km*] [-*np*]**
- *Optimizing principle*:
  - one test case for each *NV* equivalence class
  - each equivalence class covered by *at least one* test case

i. -c5 –np
ii. xxyy -c3 (no file *xxyy* in directory)
iii. #%$file5.3
iv. myfile -c0 (file *myfile* is in directory)
v. myfile -f100N
vi. myfile -f2H
vii. myfile -c5 -f1 -hjk

Test Case x Equivalence Class

TC/EC	i	ii	iii	iv	v	vi	vii
1				+	+	+	+
2	-						
3		-					
4			-				
5	+	+					+
6			+		+	+	
7				-			
8						+	+
9	+	+	+	+			
10					+		
11	+	+	+	+			+
12					-		
13						-	
14	+						
15		+	+	+	+	+	
16							-

Extending Principle

- combinations over the *number* of parameters
  - name of existing file always given
  - a test case where all the parameters are missing (0 present)
  - a test case for each individual parameter (1 present)
  - each parameter included in the set of pairs (2 present)
  - each parameter included in the set of triplets (3 present)
  - all the parameters given (4 present)

Test Case x Equivalence Class

TC/EC	viii	ix	x	xi	xii	xiii	xiv	xv	xvi	xvii
1	+	+	+	+	+	+	+	+	+	+
2										
3										
4										
5		+					+	+	+	+
6	+		+	+	+	+				
7										
8			+			+		+		+
9	+	+		+	+		+		+	
10				+		+			+	+
11	+	+	+		+		+	+		
12										
13										
14					+		+	+	+	+
15	+	+	+	+		+				
16										

Test Case x Equivalence Class

TC/EC	i	ii	iii	iv	v	vi	vii
1				+	+	+	+
2	-						
3		-					
4			-				
5	+	+					+
6			+		+	+	
7				-			
8						+	+
9	+	+	+	+			
10					+		
11	+	+	+	+			+
12					-		
13						-	
14	+						
15		+	+	+	+	+	
16							-

Example

- ***print* file [-cn] [-fkm] [-np]**

viii. myfile (none present)
ix. myfile –c1 (n present)
x. myfile –f9 (k present)
xi. myfile –fB (m present)
xii. myfile –np (-np present)
xiii. myfile –f1N (k, m present)
xiv. myfile –c100 –np (n, -np present)
xv. myfile –c50 –f5 –np (n, k, -np present)
xvi. myfile –c1 –fB –np (n, m, -np present)
xvii. myfile –c99 –f2N –np (all present)

References

[1] Myers, *The Art of Software Testing, 1978*

WHITE BOX TESTING

Week 10

White Box Testing

0 + 1 + 2 + … + *i*, *i* ∈ [0, 100]:

- read(i);
- **if** ((i < 0) || (i > 100))
  - error();
- **else**
  - { sum=0; x=0;
  - **while** (x < i)
    - { x=x+1;
    - **if** (i==10) sum=1; **else** sum=sum+x; }
  - print(sum); }

Black box test cases

- *i* = -1 OK
- *i* = 0 OK
- *i* = 1 OK
- *i* = 50 OK
- *i* = 99 OK
- *i* = 100 OK
- *i* = 101 OK
- *However*:
- *i* = 10 ⇒ ***FAILURE!*** **(sum=1)**

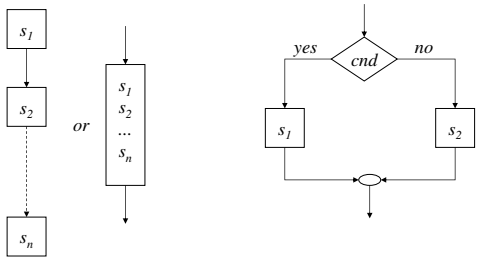
White-box testing principles

- ***(control) flow graph*****:** abstraction of the program's control flow, in graphical form
- ***data-flow graph*****:** abstraction of the program's data flow (for a certain input variable), in graphical form; usually extension of control-flow graph
- control-flow graph, data-flow graph automatically produced
- test cases designed from the graphs
- ***coverage*****:** the relative amount of statements (and others) executed during testing, computed from control-flow/data-flow graph

White-box testing principles

- details of source code analyzed
- design of test cases on the basis of code structure
- ***execution path*****:** a certain sequence of program statements, executed when starting the program with a certain input (test case)
- different test cases => different execution paths
- ***control-flow testing*****:** based on the execution order of the statements
- ***data-flow testing*****:** based on the processing of the data during execution

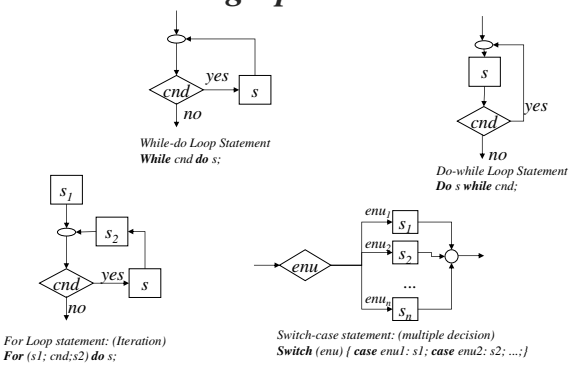
Flow graph structures



Statements sequence $s_1; s_2; ...; s_n$;

Conditional (if) statement: if cnd s_1 *else* s_2.

Flow graph structures



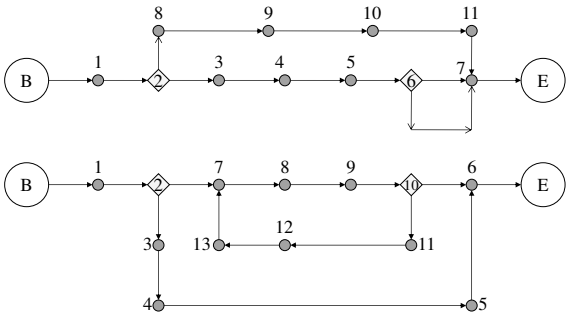
Control-flow testing

- ***Coverage***: how extensively the program has been (or will be) tested with a given set of test cases
- the (relative) number of *nodes* (statements) in the flow graph executed during testing
- the (relative) number of *edges* (control transitions) in the flow graph traversed during testing
- ***Statement coverage***: each node (statement) has to be executed at least once
- ***Branch coverage***: each edge (transition) has to be traversed at least once
- a large number of variations of different coverage power
- special target: ***loop testing***

Execution Path

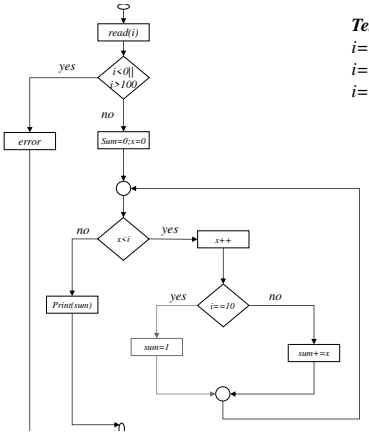
- a *sequence of nodes and connecting edges* from the unique begin-node of the flow graph to the unique end-node of the graph.
- A ***certain instance of the relevant program execution***
- May contain the same node several times: *loops*.

White-box (structural) testing



Statement coverage criterion

- A set *P* of execution paths satisfies the ***statement coverage criterion*** if and only if for all *nodes n* in the flow graph, there is at least one path *p* in *P* such that *p* contains the node *n* ≡ Each statement of the program is executed at least once during testing, by some test case.
  - criterion met => *complete* (100%) statement coverage
  - criterion not met => partial statement coverage (< 100%)
  - begin-node, end-node, junctions excluded
  - complete coverage surprisingly hard to achieve in practice
  - "dead code" / conditional compilation



***Test cases**:*
i=-1; i=0; i=1;
i=50; i=99;
i=100; i=101

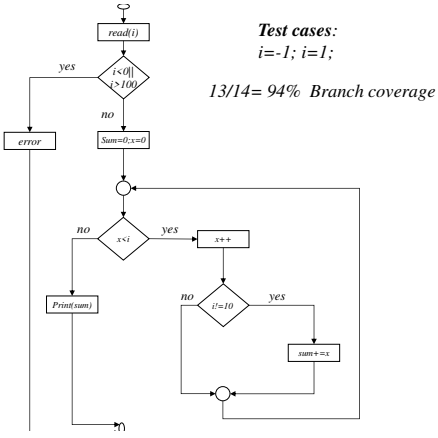
Statement Coverage:
9/10=90%

Branch coverage:
13/15=87%

***red-lined statement is not executed!**

Statement coverage ≠ Branch coverage

- read(i);
- **if** ((i < 0) || (i > 100)) error() **else**
- { sum=0; x=0;
  - **while** (x < i)
  - { x=x+1; **if** (i <> 10) sum=sum+x; }
  - print(sum);
- }



***Test cases**:*
i=-1; i=1;

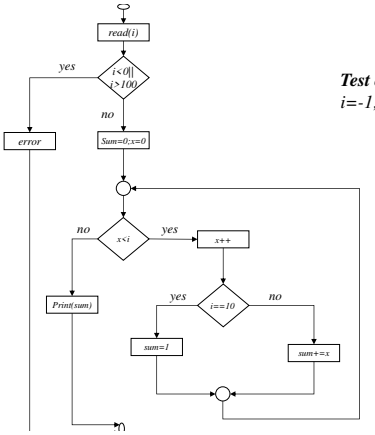
13/14= 94% Branch coverage

Multicondition coverage criterion

- A set *P* of execution paths satisfies the ***multicondition coverage criterion*** if and only if for every control node in the flow graph consisting of atomic predicates (c_1, c_2, …, c_n), each possible *combination* of their truth values (*true/yes*, *false/no*) is evaluated within at least one path *p* in *P*.
  - stronger requirement than for condition coverage all the *combinations*
  - for 2 atomic predicates: (*true*,*true*), (*true*,*false*), (*false*,*true*), (*false*,*false*)
  - for 3 atomic predicates: 8 combinations, etc….

Branch coverage criterion

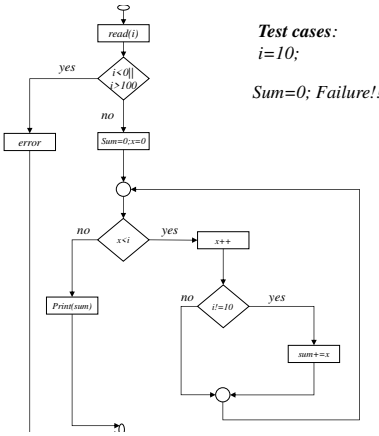
- A set *P* of execution paths satisfies the ***branch coverage criterion*** if and only if for all *edges e* in the flow graph, there is at least one path *p* in *P* such that *p* contains the edge *e* ≡ Each control-flow branch / decision (*true / yes*, *false / no*) is taken at least once during testing, by some test case.
  - criterion met => *complete* (100%) branch coverage
  - complete branch coverage => complete statement coverage (branch coverage *subsumes* statement coverage)
  - usually more tests are needed for complete branch coverage than needed for complete statement coverage
  - branch coverage is more extensive: the criterion is stronger than the statement coverage criterion
  - criterion not met => partial branch coverage (< 100%)



***Test cases**:*
i=-1; i=1; i=10;

Statement Coverage:
10/10=100%

Branch coverage:
15/15=100%



***Test cases**:*
i=10;

Sum=0; Failure!!!

Condition coverage criterion

- A set *P* of execution paths satisfies the ***condition coverage criterion*** if and only if for every control node in the flow graph consisting of atomic predicates (c_1, c_2, …, c_n), c_i yields *true* (*yes*) when evaluated within a path p_1 in *P* and c_i yields *false* (*no*) when evaluated within a path p_2 in *P*, $i = 1, 2, \ldots, n$.
  - internal structure of composite control predicates taken into account: (i < 0) || (i > 100)
  - each predicate ("i<0," "i>100") tested separately for both "true" and "false."

Path coverage criterion

- Path coverage the strongest criterion
  - usually impossible to reach

Independent path coverage criterion.

- Independent path coverage stronger than branch coverage
  - used also in complexity analysis of programs

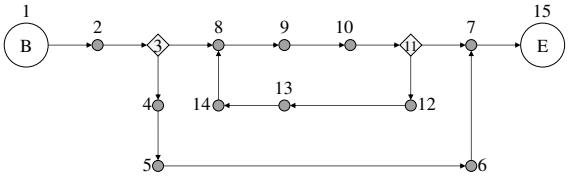
Cyclomatic Complexity

- Given that the source code is represented as a directed graph (i.e., a control-flow graph), CC can be formulated as

$$CC = E - N + 2$$

  - where
    - E is the number of edges and
    - N is the number of nodes
  - of the graph.

Cyclomatic Complexity



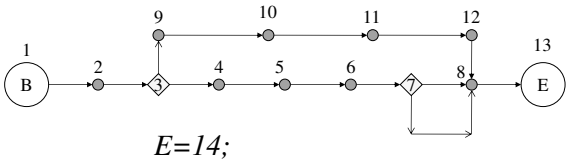
E=16;
N=15;
CC=E-N+2=3
CC=3

Independent paths
1 2 3 8 9 10 11 7 15
1 2 3 4 5 6 7 15
1 2 3 8 9 10 11 12 13 14 8 9 10 11 7 15

Cyclomatic Complexity

- ***Cyclomatic complexity (CC)*** of a piece of code is the *number of linearly independent (LI) paths* through the piece of code.
- ***E.g.*** For a piece of code *with no decision structures*, ***CC=1***, meaning there is a *single* LI path.

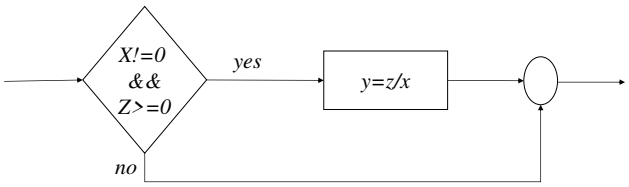
Cyclomatic Complexity



E=14;
N=13;
CC=E-N+2=3
CC=3

An example for coverage comparison

if ((x != 0) && (z >= 0)) y=z/x ;



Comparison of coverages

- Complete statement coverage: ($x = 1$, $z = 0$) **[1 test]**
- Complete branch coverage: ($x = 1$, $z = 0$) for the *yes* branch, ($x = 1$, $z = -1$) for the *no*-branch **[2 tests]**
- Complete condition coverage: ($x = 0$, $z = 0$) for combination (*false*, *true*), ($x = 1$, $z = -1$) for combination (*true*, *false*) (*yes*-branch unexplored !) **[2 tests]**
- Complete multicondition coverage: ($x = 1$, $z = 0$) for combination (*true*, *true*), ($x = 1$, $z = -1$) for combination (*true*, *false*), ($x = 0$, $z = 0$) for the combination (*false*, *true*), ($x = 0$, $z = -1$) for the combination (*false*, *false*) **[4 tests]**

Selection of test cases

- A certain code coverage criterion and percentage (e.g. 100% branch coverage) has been chosen ⇒ one has to design test cases to reach the criterion.

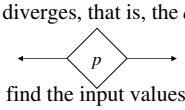
1) Construct a flow graph for the program (with a white-box tool).
2) Choose the execution paths that satisfy the criterion.
3) For each execution path, design a test case (program input-output) that activates a traversal of the path.
4) Execute the tests (with the white-box tool that calculates the coverage).
5) If the required coverage has not been reached, return to step 3 to design additional test cases for those execution paths that have not been traversed yet during testing.

Path sensitization

- The process of designing a test case for a particular execution path
  - In general, *path sensitization* is undecidable: there is no algorithm that can find a suitable test case for each possible path.
  - Symbolic execution and equation solving tools succeed in some cases.
  - A *heuristic*: Begin with the control conditions of a branch at the *end* of the path. Select such variable values that will satisfy these conditions. Repeat this analysis for each prior branch in the path until you reach the entry node of the flow graph. Use the selected values of the input variables as the test case for the path.
  - There may be infeasible paths that cannot be executed with any input, caused by short-circuit evaluation, contradictory or mutually exclusive control conditions, redundant control predicates, or dead code

Path Sensitization.. Cont'd

- the crucial points in a flow-graph are those where the execution diverges, that is, the ***control predicates*** of branches



- one has to find the input values such that when executing the program with the input, control branches into the desired direction and the predicate *p* obtains the corresponding value (*true / false*) or value combination
  - *note* 1: *p* may depend on the input just indirectly
  - *note* 2: it may not be possible to obtain all the required truth values for *p*: ((x == 1) && (x==2))

Loop Testing

- *Testing of simple loops*: 0 iterations (no looping), *minimum* number of iterations (possibly 0), *minimum*+1 iterations, *typical* number of iterations, *maximum*-1 iterations, *maximum* number of iterations, *maximum*+1 iterations (should not be feasible)
  - note: loops with fixed iteration control may not be executable (testable) with all the suggested iteration patterns
    for (j=0; j < 999; ++j) { … }
- *Testing of serial loops*:
  - if there is no data-flow relationship between the loops, test them both as simple loops
  - if there is a data-flow relationship between the loops, test them as if the loops were nested
- *Testing of unstructured ("spaghetti") loops*: test the loop with an equivalent simple / serial / nested loop as model
  - spaghetti code should be rewritten into structured form, for testing as well as for maintenance purposes

Loop Testing ...2

- *Testing of nested loops*:
  - There would be too many tests when repeating all the inner loop tests every time an outer loop is iterated, so:
  1) The innermost loop is tested first using the simple-loop strategy. The other loops are iterated their minimum number of times.
  2) Set up the looping conditions of the previously tested loop such that it will be iterated a suitable number of times (minimum, typical, or maximum).
  3) Proceed to testing the outer loop which is nesting the previously tested one, using the simple-loop strategy. (The outer loops are iterated their minimum number of times, the inner loops are iterated their suitable number of times.)
  4) Repeat the steps 2 and 3, until the outermost loop has been tested.
  5) Set up a test that will iterate all loops their maximum number of times.

References

[1] Myers, *The Art of Software Testing*, 1978

Software cost estimation

Topics covered

- Software productivity
- Estimation techniques
- Algorithmic cost modelling
- Project duration and staffing

Fundamental estimation questions

- How much effort is required to complete an activity?
- How much calendar time is needed to complete an activity?
- What is the total cost of an activity?
- Project estimation and scheduling are interleaved management activities.

Software cost components

- Hardware and software costs.
- Travel and training costs.
- Effort costs (the dominant factor in most projects)
  - The salaries of engineers involved in the project;
  - Social and insurance costs.
- Effort costs must take overheads into account
  - Costs of building, heating, lighting.
  - Costs of networking and communications.
  - Costs of shared facilities (e.g library, staff restaurant, etc.).

Costing and pricing

- Estimates are made to discover the cost, to the developer, of producing a software system.
- There is not a simple relationship between the development cost and the price charged to the customer.
- Broader organisational, economic, political and business considerations influence the price charged.

Software pricing factors

Market opportunity	A development organisation may quote a low price because it wishes to move into a new segment of the soft ware market. Accepting a low profit on one project may give the opportunity of more profit later. The experience gained may allow new products to be developed.
Cost estimate uncertainty	If an o rganisation is unsure of its cost estimate, it may increase its price by some contingency over and above its normal profit.
Contractual terms	A c ustomer may be willing to allow the developer to retain ownership of the source code and reuse it in other projects. T he price charged may then be less than if the soft ware source code is handed over to the customer.
Requirements volatility	If the requirements are likely to change, an organisation may lower its price to win a contract. After the contract is awarded, high prices can be charged fo r changes to the requirements.
Financial health	Developers in financial difficulty may lower their price to gain a contract. It is better to make a smaller than normal profit or break even than to go out of business.

Software productivity

- A measure of the rate at which individual engineers involved in software development produce software and associated documentation.
- Not quality-oriented although quality assurance is a factor in productivity assessment.
- Essentially, we want to measure useful functionality produced per time unit.

Productivity measures

- Size related measures based on some output from the software process. This may be lines of delivered source code, object code instructions, etc.
- Function-related measures based on an estimate of the functionality of the delivered software. Function-points are the best known of this type of measure.

Measurement problems

- Estimating the size of the measure (e.g. how many function points).
- Estimating the total number of programmer months that have elapsed.
- Estimating contractor productivity (e.g. documentation team) and incorporating this estimate in overall estimate.

Lines of code

- What's a line of code?
  - The measure was first proposed when programs were typed on cards with one line per card;
  - How does this correspond to statements as in Java which can span several lines or where there can be several statements on one line.
- What programs should be counted as part of the system?
- This model assumes that there is a linear relationship between system size and volume of documentation.

Productivity comparisons

- The lower level the language, the more productive the programmer
  - The same functionality takes more code to implement in a lower-level language than in a high-level language.
- The more verbose (wordy) the programmer, the higher
  the productivity
  - Measures of productivity based on lines of code suggest that programmers who write verbose code are more productive than programmers who write compact code.

System development times

	Analysis	Design	Coding	Testing	Documentation
Assembly code	3 weeks	5 weeks	8 weeks	10weeks	2 weeks
High-level language	3 weeks	5 weeks	4 weeks	6 weeks	2 weeks

	Size	Effort	Productivity
Assembly code	5000 lines	28 weeks	714 lines/month
High-level language	1500 lines	20 weeks	300 lines/month

Function points

- Based on a combination of program characteristics
  - external inputs and outputs;
  - user interactions;
  - external interfaces;
  - files used by the system.
- A weight is associated with each of these and the function point count is computed by multiplying each raw count by the weight and summing all values.
- UFC: unadjusted function-point count

$$UFC = \sum(\text{number of elements of given type}) \times (\text{weight})$$

Function points

- The function point count is modified by complexity of the project
- FPs can be used to estimate LOC depending on the average number of LOC per FP for a given language
  - LOC = AVC * number of function points;
  - AVC (avg. # of LOC) is a language-dependent factor varying from 200-300 for assemble language to 2-40 for a 4GL;
- FPs are very subjective. They depend on the estimator
  - Automatic function-point counting is impossible.

Object points

- Object points (alternatively named application points) are an alternative function-related measure to function points when 4GLs or similar languages are used for development.
- Object points are NOT the same as object classes.
- The number of object points in a program is a weighted estimate of
  - The number of separate screens that are displayed;
  - The number of reports that are produced by the system;
  - The number of program modules that must be developed to supplement the database code;

Object point estimation

- Object points are easier to estimate from a specification than function points as they are simply concerned with screens, reports and programming language modules.
- They can therefore be estimated at a fairly early point in the development process.
- At this stage, it is very difficult to estimate the number of lines of code in a system.

Productivity estimates

- Real-time embedded systems, 40-160 LOC/P-month.
- Systems programs , 150-400 LOC/P-month.
- Commercial applications, 200-900 LOC/P-month.
- In object points, productivity has been measured between 4 and 50 object points/month depending on tool support and developer capability.

Factors affecting productivity

Application domain experience	Knowledge of the application domain is essential for effe ctive soft ware development. Engineers who already understand a domain are likely to be the most productive.
Process quality	The development process used can have a s ignificant eff ect on productivity. T his is covered in Chapter 28.
Project size	The larger a project, the more time required fo r team communications. Less time is available for development so individual productivity is reduced.
Technology support	Good support technology such as C ASE tools, configuration management systems, etc. can improve productivity.
Working environment	As I discussed in Chapter 25, a quiet working environment with private work areas contributes to improved productivity.

Quality and productivity

- All metrics based on volume/unit time are flawed because they do not take quality into account.
- Productivity may generally be increased at the cost of quality.
- It is not clear how productivity/quality metrics are related.
- If requirements are constantly changing then an approach based on counting lines of code is not meaningful as the program itself is not static;

Estimation techniques

- There is no simple way to make an accurate estimate of the effort required to develop a software system
  - Initial estimates are based on inadequate information in a user requirements definition;
  - The software may run on unfamiliar computers or use new technology;
  - The people in the project may be unknown.
- Project cost estimates may be self-fulfilling
  - The estimate defines the budget and the product is adjusted to meet the budget.

Changing technologies

- Changing technologies may mean that previous estimating experience does not carry over to new systems
  - Distributed object systems rather than mainframe systems;
  - Use of web services;
  - Use of ERP or database-centred systems;
  - Use of off-the-shelf software;
  - Development for and with reuse;
  - Development using scripting languages;
  - The use of CASE tools and program generators.

Estimation techniques

- Algorithmic cost modelling.
- Expert judgement.
- Estimation by analogy.
- Parkinson's Law.
- Pricing to win.

Estimation techniques

Algorithmic cost modelling	A model based on historical cost info rmation that relates some software metric (usually its size) to the project cost is used. An estimate is made of that metric and the model predicts the eff ort required.
Expert judgement	Several experts on the proposed software development techniques and the application domain are consulted. T hey each estimate the project cost. These estimates are compared and discussed. T he estimation process iterates until an agreed estimate is reached.
Estimation by analogy	This technique is applicable when other projects in the same application domain have been completed. T he cost of a new project is estimated by analogy with these completed projects. Myers (Myers 1989) gives a very clear description of this approach.
Parkinson's Law	Parkinson's Law states that work expands to fill the time available. The cost is determined by available resources rather than by objective assessment. If the software has to be delivered in 12 months and 5 people are available, the effo rt required is estimated to be 60 person-months.
Pricing to win	The software cost is estimated to be whatever the customer has available to spend on the project. The estimated effo rt depends on the customer's budget and not on the software functionality.

Pricing to win

- The project costs whatever the customer has to spend on it.
- Advantages:
  - You get the contract.
- Disadvantages:
  - The probability that the customer gets the system he or she wants is small. Costs do not accurately reflect the work required.

Top-down and bottom-up estimation

- Any of these approaches may be used top-down or bottom-up.
- Top-down
  - Start at the system level and assess the overall system functionality and how this is delivered through sub-systems.
- Bottom-up
  - Start at the component level and estimate the effort required for each component. Add these efforts to reach a final estimate.

Top-down estimation

- Usable without knowledge of the system architecture and the components that might be part of the system.
- Takes into account costs such as integration, configuration management and documentation.
- Can underestimate the cost of solving difficult low-level technical problems.

Bottom-up estimation

- Usable when the architecture of the system is known and components identified.
- This can be an accurate method if the system has been designed in detail.
- It may underestimate the costs of system level activities such as integration and documentation.

Estimation methods

- Each method has strengths and weaknesses.
- Estimation should be based on several methods.
- If these do not return approximately the same result, then you have insufficient information available to make an estimate.
- Some action should be taken to find out more in order to make more accurate estimates.
- Pricing to win is sometimes the only applicable method.

Pricing to win

- This approach may seem unethical and un-businesslike.
- However, when detailed information is lacking it may be the only appropriate strategy.
- The project cost is agreed on the basis of an outline proposal and the development is constrained by that cost.
- A detailed specification may be negotiated or an evolutionary approach used for system development.

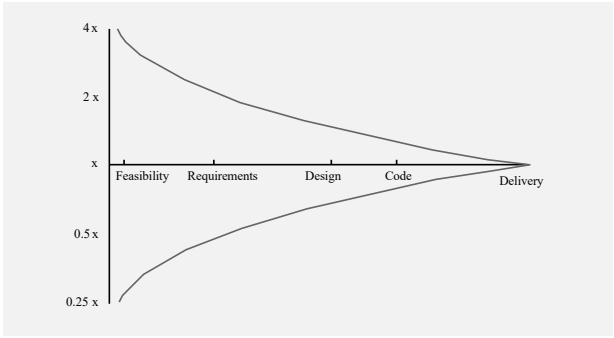
Algorithmic cost modelling

- Cost is estimated as a mathematical function of product, project and process attributes whose values are estimated by project managers:
  - Effort = A $\times$ Size^{B} $\times$ M
  - A is an organisation-dependent constant, B reflects the disproportionate effort for large projects and M is a multiplier reflecting product, process and people attributes.
- The most commonly used product attribute for cost estimation is code size.
- Most models are similar but they use different values for A, B and M.

Estimation accuracy

- The size of a software system can only be known accurately when it is finished.
- Several factors influence the final size
  - Use of COTS and components;
  - Programming language;
  - Distribution of system.
- As the development process progresses then the size estimate becomes more accurate.

Estimate uncertainty



The COCOMO model

- An empirical model based on project experience.
- Well-documented, 'independent' model which is not tied to a specific software vendor.
- Long history from initial version published in 1981 (COCOMO-81) through various instantiations to COCOMO 2.
- COCOMO 2 takes into account different approaches to software development, reuse, etc.

COCOMO 81

Project complexity	Formula	Description
Simple	PM = 2.4 (KDSI)$^{1.05}$ $\times$ M	Well-understood applications developed by small teams.
Moderate	PM = 3.0 (KDSI)$^{1.12}$ $\times$ M	More complex projects where team members may have limited experience of related systems.
Embedded	PM = 3.6 (KDSI)$^{1.20}$ $\times$ M	Complex projects where the software is part of a strongly coupled complex of hardware, soft ware, regulations and operational procedures.

COCOMO 2

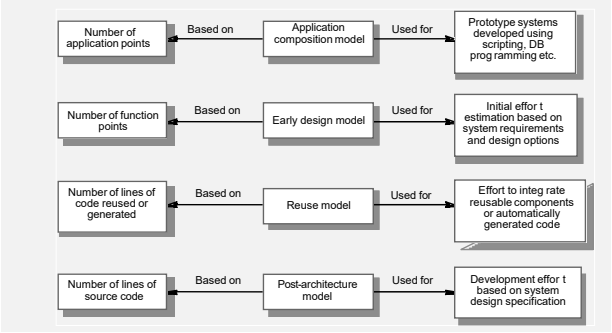
COCOMO 81 was developed with the assumption that a waterfall process would be used and that all software would be developed from scratch.

Since its formulation, there have been many changes in software engineering practice and COCOMO 2 is designed to accommodate different approaches to software development.

COCOMO 2 models

- COCOMO 2 incorporates a range of sub-models that produce increasingly detailed software estimates.
- The sub-models in COCOMO 2 are:
  - Application composition model. Used when software is composed from existing parts.
  - Early design model. Used when requirements are available but design has not yet started.
  - Reuse model. Used to compute the effort of integrating reusable components.
  - Post-architecture model. Used once the system architecture has been designed and more information about the system is available.

Use of COCOMO 2 models



Application composition model

- Supports prototyping projects and projects where there is extensive reuse.
- Based on standard estimates of developer productivity in application (object) points/month.
- Takes CASE tool use into account.
- Formula is
  - PM = (NAP × (1 - %reuse/100)) / PROD
  - PM is the effort in person-months, NAP is the number of application points and PROD is the productivity.

Object point productivity

Developer's experience and capability	Very low	Low	Nominal	High	Very high
ICASE maturity and capability	Very low	Low	Nominal	High	Very high
PROD (NOP/month)	4	7	13	25	50

Early design model

- Estimates can be made after the requirements have been agreed.
- Based on a standard formula for algorithmic models
  - PM = A × Size^{B} × M where
  - M = PERS × RCPX × RUSE × PDIF × PREX × FCIL × SCED;
  - A = 2.94 in initial calibration, Size in KLOC, B varies from 1.1 to 1.24 depending on novelty of the project, development flexibility, risk management approaches and the process maturity.

Multipliers

Multipliers reflect the capability of the developers, the non-functional requirements, the familiarity with the development platform, etc.

RCPX - product reliability and complexity;
RUSE - the reuse required;
PDIF - platform difficulty;
PREX - personnel experience;
PERS - personnel capability;
SCED - required schedule;
FCIL - the team support facilities.

The reuse model

Takes into account black-box code that is reused without change and code that has to be adapted to integrate it with new code.

There are two versions:

Black-box reuse where code is not modified. An effort estimate (PM) is computed.

White-box reuse where code is modified. A size estimate equivalent to the number of lines of new source code is computed. This then adjusts the size estimate for new code.

Reuse model estimates 1

For generated code:

PM = (ASLOC * AT/100)/ATPROD

ASLOC is the number of lines of generated code

AT is the percentage of code automatically generated.

ATPROD is the productivity of engineers in integrating this code.

Reuse model estimates 2

When code has to be understood and integrated:

ESLOC = ASLOC * (1-AT/100) * AAM.

ASLOC and AT as before.

AAM is the adaptation adjustment multiplier computed from the costs of changing the reused code, the costs of understanding how to integrate the code and the costs of reuse decision making.

Post-architecture level

Uses the same formula as the early design model but with 17 rather than 7 associated multipliers.

The code size is estimated as:

Number of lines of new code to be developed;

Estimate of equivalent number of lines of new code computed using the reuse model;

An estimate of the number of lines of code that have to be modified according to requirements changes.

The exponent term

This depends on 5 scale factors (see next slide). Their sum/100 is added to 1.01

A company takes on a project in a new domain. The client has not defined the process to be used and has not allowed time for risk analysis. The company has a CMM level 2 rating.

Precedentedness - new project (4)
Development flexibility - no client involvement - Very high (1)
Architecture/risk resolution - No risk analysis - V. Low .(5)
Team cohesion - new team - nominal (3)
Process maturity - some control - nominal (3)

Scale factor is therefore 1.17.

Exponent scale factors

Precedentedness	Reflects the previous experience of the organisation with this type of project. Very low means no previous experience, Extra high means that the organisation is completely familiar with this application domain.
Development flexibility	Reflects the degree of flexibility in the development process. Very low means a prescribed process is used; Extra high means that the client only sets general goals.
Architecture/risk resolution	Reflects the extent of risk analysis carried out. Very low means little analysis, Extra high means a complete a thorough risk analysis.
Team cohesion	Reflects how well the development team know each other and work together. Very low means very difficult interactions, Extra high means an integrated and effective team with no communication problems.
Process maturity	Reflects the process maturity of the organisation. The computation of this value depends on the CMM Maturity Questionnaire but an estimate can be achieved by subtracting the CMM process maturity level from 5.

Multipliers

Product attributes

Concerned with required characteristics of the software product being developed.

Computer attributes

Constraints imposed on the software by the hardware platform.

Personnel attributes

Multipliers that take the experience and capabilities of the people working on the project into account.

Project attributes

Concerned with the particular characteristics of the software development project.

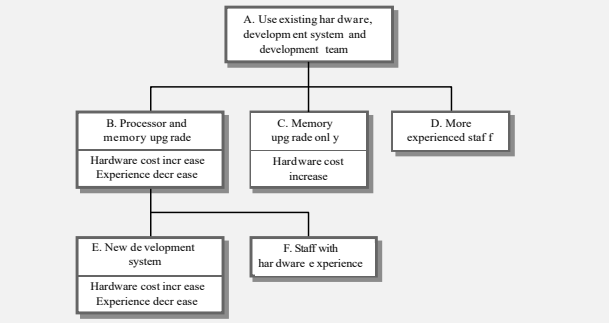
Effects of cost drivers

Exponent value	1.17
System size (including factors for reuse and requirements volatility)	128, 000 DSI
Initial COCOMO estimate without cost drivers	**730 person-months**
Reliability	Very high, multiplier = 1.39
Complexity	Very high, multiplier = 1.3
Memory constraint	High, multiplier = 1.21
Tool use	Low, multiplier = 1.12
Schedule	Accelerated, multiplier = 1.29
Adjusted COCOMO estimate	**2306 person-months**
Reliability	Very low, multiplier = 0.75
Complexity	Very low, multiplier = 0.75
Memory constraint	None, multiplier = 1
Tool use	Very high, multiplier = 0.72
Schedule	Normal, multiplier = 1
Adjusted COCOMO estimate	**295 person-months**

Project planning

- Algorithmic cost models provide a basis for project planning as they allow alternative strategies to be compared.
- Embedded spacecraft system
  - Must be reliable;
  - Must minimise weight (number of chips);
  - Multipliers on reliability and computer constraints > 1.
- Cost components
  - Target hardware;
  - Development platform;
  - Development effort.

Management options



Management option costs

Option	RELY	STOR	TIME	TOOLS	LTEX	Total effort	Software cost	Hardware cost	Total cost
A	1.39	1.06	1.11	0.86	1	63	949393	100000	1049393
B	1.39	1	1	1.12	1.22	88	1313550	120000	1402025
C	1.39	1	1.11	0.86	1	60	895653	105000	1000653
D	*1.39*	*1.06*	*1.11*	*0.86*	*0.84*	*51*	*769008*	*100000*	*897490*
E	1.39	1	1	0.72	1.22	56	844425	220000	1044159
F	1.39	1	1	1.12	0.84	57	851180	120000	1002706

Option choice

- Option D (use more experienced staff) appears to be the best alternative
  - However, it has a high associated risk as experienced staff may be difficult to find.
- Option C (upgrade memory) has a lower cost saving but very low risk.
- Overall, the model reveals the importance of staff experience in software development.

Project duration and staffing

- As well as effort estimation, managers must estimate the calendar time required to complete a project and when staff will be required.
- Calendar time can be estimated using a COCOMO 2 formula
  - $TDEV = 3 \times (PM)^{(0.33+0.2*(B-1.01))}$
  - PM is the effort computation and B is the exponent computed as discussed above (B is 1 for the early prototyping model). This computation predicts the nominal schedule for the project.
- The time required is independent of the number of people working on the project.

Staffing requirements

- Staff required can't be computed by diving the development time by the required schedule.
- The number of people working on a project varies depending on the phase of the project.
- The more people who work on the project, the more total effort is usually required.
- A very rapid build-up of people often correlates with schedule slippage.

Key points

- There is not a simple relationship between the price charged for a system and its development costs.
- Factors affecting productivity include individual aptitude, domain experience, the development project, the project size, tool support and the working environment.
- Software may be priced to gain a contract and the functionality adjusted to the price.

Key points

Different techniques of cost estimation should be used when estimating costs.

The COCOMO model takes project, product, personnel and hardware attributes into account when predicting effort required.

Algorithmic cost models support quantitative option analysis as they allow the costs of different options to be compared.

The time to complete a project is not proportional to the number of people working on the project.